

GuichetIA

Assistant administratif intelligent pour le Burkina Faso

Rapport Technique

Projet	Data Science 2026 — Option 7 : Assistant Juridique et Administratif
Groupe	OUEDRAOGO Zeïd El Gazeli, DICKO Mamoudou, OUEDRAOGO Abdoul Kader W. Ghislan
Classe	Master 1 Data Science IFOAD
Soutenance	Lundi 13 juillet 2026
GitHub	github.com/Odg75/projet_data_science_GuichetIA
Frontend	projet-data-science-guichet-ia.vercel.app
Backend	guichetia-backend.onrender.com

Table des matières

- 1. Introduction et contexte
- 2. Architecture du système RAG
- 3. Corpus et stratégie d'ingestion
- 4. Modèles retenus
- 5. Interface et déploiement
- 6. Évaluation
- 7. Limites et perspectives
- 8. Conclusion

1. Introduction et contexte

GuichetIA est un assistant administratif basé sur une architecture RAG (Retrieval-Augmented Generation) conçu pour répondre aux questions des citoyens du Burkina Faso sur les démarches administratives courantes. Le projet s'inscrit dans l'option 7 du sujet Projet Data Science 2026 proposé par le Dr Delwende D. Arthur Sawadogo dans le cadre du Master 1 Data Science IFOAD.

L'objectif central n'est pas de produire un chatbot conversationnel générique, mais un agent capable de rechercher l'information pertinente dans une base de connaissances locale et vérifiée, de formuler une réponse sourcée, et de reconnaître explicitement ses limites lorsqu'une question est hors de son périmètre (plutôt que d'inventer une procédure, un montant ou un délai).

1.1 Problématique

Les démarches administratives burkinabè (CNIB, passeport, création d'entreprise, casier judiciaire, acte de naissance, certificat de nationalité) sont documentées sur des portails officiels qui restent peu accessibles pour des citoyens n'ayant pas nécessairement accès à internet ou habitués aux recherches numériques. GuichetIA propose un point d'entrée unique en langage naturel, capable de répondre avec précision à partir de sources officielles vérifiées.

1.2 Périmètre couvert

Démarche	Informations couvertes	Source
CNIB	Pièces, coût (2 500 F CFA), délai, perte, renouvellement	oni.bf, police.gov.bf
Passeport	Pièces, coût (50 000 F CFA), délai 72h, renouvellement	oni.bf, police.gov.bf
Création SARL/EI	RCCM, IFU, CPC, CNSS, CEFORE/MEBF, délais	service-public.gov.bf
Casier judiciaire	Types, lieu de dépôt, délai, coût	service-public.gov.bf
Acte de naissance	Copies, jugement supplétif, état civil	service-public.gov.bf
Cert. nationalité	Pièces, demande, validité	service-public.gov.bf

Tableau 1 — Les six démarches couvertes par GuichetIA.

2. Architecture du système RAG

GuichetIA adopte une architecture RAG (Retrieval-Augmented Generation) en deux phases distinctes : une phase hors ligne (ingestion et indexation des documents sources) et une phase en ligne (interrogation, retrieval et génération de réponse).

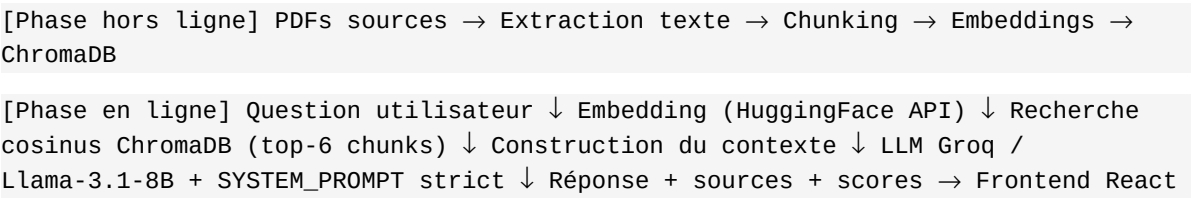


Figure 1 — Flux de données du pipeline RAG GuichetIA.

2.1 Composants principaux

Composant	Rôle	Outil
Ingestion	Extraction texte PDF, chunking, indexation	PyMuPDF + LangChain
Base vectorielle	Stockage et recherche par similarité cosinus	ChromaDB (persistant)
Embeddings (index)	Vectorisation des chunks à l'indexation	HF Inference API
Embeddings (query)	Vectorisation de la question entrante	HF Inference API
LLM	Génération de la réponse grounded	Groq / Llama-3.1-8B
Orchestration	Chaining retrieval + génération	LangChain
Backend API	Endpoints /health et /ask	FastAPI (Python)
Frontend	Interface de chat utilisateur	React + Tailwind CSS

Tableau 2 — Composants de l'architecture GuichetIA.

2.2 Prompt système et guardrails

Un SYSTEM_PROMPT strict est fourni au LLM à chaque appel. Il impose trois règles absolues : (1) répondre uniquement à partir du contexte récupéré, (2) répondre par une formule de repli explicite si l'information est absente du contexte, (3) ne jamais citer d'URL ou de liens dans la réponse (l'interface affiche les sources automatiquement). Cette approche garantit que le LLM reste un générateur de langage contrôlé, sans connaissance hors-corpus.

3. Corpus et stratégie d'ingestion

3.1 Construction du corpus

Aucun PDF officiel unique et téléchargeable ne couvre intégralement chaque démarche. Le contenu a donc été vérifié manuellement sur les portails officiels burkinabè, puis compilé en 6 PDFs sources (un par démarche), chacun citant explicitement son URL d'origine. Ces PDFs sont versionnés dans le dépôt git (backend/data/pdfs/) pour garantir la reproductibilité de l'indexation.

Fichier PDF	Démarche	Source officielle
cnib.pdf	CNIB	oni.bf + police.gov.bf
passport.pdf	Passeport	oni.bf + police.gov.bf
creation_entreprise.pdf	Création d'entreprise	service-public.gov.bf (PP + PM)
casier_judiciaire.pdf	Casier judiciaire	service-public.gov.bf
acte_naissance.pdf	Acte de naissance	service-public.gov.bf
certificat_nationalite.pdf	Cert. de nationalité	service-public.gov.bf

Tableau 3 — Les 6 PDFs sources du corpus GuichetIA.

3.2 Pipeline d'ingestion

Le pipeline (backend/ingestion/build_index.py) suit quatre étapes :

- **Extraction** : PyMuPDF extrait le texte brut de chaque PDF source (extract_pdf.py). Les métadonnées (démarche, source_url) sont préservées dans les documents LangChain.
- **Chunking** : RecursiveCharacterTextSplitter (LangChain) découpe chaque document en segments de 500 caractères avec un recouvrement de 50 caractères. Les séparateurs utilisés sont (dans l'ordre) : paragraphes doubles, sauts de ligne, points, espaces. Résultat : 32 chunks sur l'ensemble du corpus.
- **Embeddings** : Chaque chunk est vectorisé via l'API d'inférence HuggingFace (modèle paraphrase-multilingual-MiniLM-L12-v2), sans chargement local du modèle — une contrainte imposée par la limite RAM de 512 Mo du plan Render gratuit. Les vecteurs sont normalisés (norme L2 = 1) avant indexation pour garantir la cohérence de la métrique cosinus.
- **Indexation** : Les vecteurs normalisés sont persistés dans ChromaDB (data/chroma_db/) avec la métrique cosinus (hnsw:space=cosine), rejouable à l'identique en relançant le script.

Lors de l'interrogation (phase en ligne), la question de l'utilisateur est vectorisée de la même façon (même modèle, même normalisation, via HF Inference API), puis une recherche par **similarité cosinus** retourne les top-6 chunks (TOP_K = 6). Les distances cosinus retournées par ChromaDB (0 = identique, 1 = opposé) sont converties en scores de pertinence 0–100 via la formule : **score = (1 – distance) × 100**.

4. Modèles retenus

4.1 Modèle d'embeddings

Attribut	Valeur
Modèle	sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
Fournisseur	HuggingFace (Inference API — gratuit)
Dimension	384
Langues	50+ langues dont français et langues africaines
Usage en production	API distante (évite le chargement torch en local)
Justification	Multilingue, léger, gratuit, supporte le français administratif

Tableau 4 — Caractéristiques du modèle d'embeddings.

Le choix d'un modèle généraliste multilingue plutôt qu'un modèle spécialisé en français (comme CamemBERT) a été guidé par la contrainte de latence et de disponibilité via API. Un modèle spécialisé pourrait améliorer la précision du retrieval sur du français administratif, mais n'est pas disponible sans hébergement dédié.

4.2 LLM de génération

Attribut	Valeur
Modèle	llama-3.1-8b-instant
Fournisseur	Groq (API gratuite)
Paramètres	8 milliards
Température	0.1 (réponses factuelles, peu de variabilité)
Latence typique	500–1500 ms (génération), 300–800 ms (retrieval)
Justification	Gratuit, très rapide (Groq LPU), performant en français

Tableau 5 — Caractéristiques du LLM de génération.

La température 0.1 (quasi-déterministe) est choisie délibérément : pour un assistant administratif, la reproductibilité des réponses et la fidélité au contexte priment sur la créativité. Le modèle Llama 3.1 démontre de bonnes capacités en français, suffisantes pour reformuler le contenu officiel de façon claire et structurée.

5. Interface et déploiement

5.1 Backend (FastAPI)

Le backend expose trois endpoints via FastAPI (Python) :

- **GET /health** : retourne l'état de disponibilité de l'index ChromaDB et du LLM (`{"status":"ok","index_ready":true,"llm_ready":true}`). Utile pour le monitoring Render.
- **POST /ask** : reçoit la question, exécute le pipeline RAG, et retourne la réponse avec les chunks, les scores de pertinence cosinus, les sources et les temps d'exécution.
- **GET /pdfs/{démarche}** : sert le PDF source d'une démarche (accès direct au document officiel).

Le modèle de réponse AskResponse inclut : answer, sources (liste), scores (liste 0–100), score_moyen, chunks (texte + démarche + score), search_time_ms, gen_time_ms, top_k, llm_model, suggested_questions.

5.2 Frontend (React + Tailwind)

L'interface utilisateur est construite en React avec Tailwind CSS. Elle comprend :

- **Landing page** : présentation de GuichetIA, accès direct aux 6 démarches.
- **Chat Dashboard** : interface de conversation avec suggestions de questions contextuelles, badges de source, indicateurs de fiabilité (bouclier vert = réponse RAG, alerte orange = hors périmètre) et liens vers les sites officiels (oni.bf, police.gov.bf, etc.).
- **Page À propos** : description de la stack technique et de l'architecture.

5.3 Déploiement

Composant	Plateforme	URL	Plan
Backend	Render	guichetia-backend.onrender.com	Gratuit (512 Mo RAM)
Frontend	Vercel	projet-data-science-guichet-ia.vercel.app	Gratuit
Code source	GitHub	github.com/Odg75/projet_data_science_GuichetIA	Public

Tableau 6 — Plateformes de déploiement.

Le backend et le frontend sont déployés et versionnés indépendamment, ce qui permet de mettre à jour l'interface sans redéployer le pipeline RAG et inversement. Les clés API (GROQ_API_KEY, HUGGINGFACEHUB_API_TOKEN) sont injectées uniquement via les variables d'environnement des plateformes, jamais committées dans le dépôt.

6. Évaluation

L'évaluation est réalisée via le script `backend/evaluation/evaluate.py` sur un jeu de 10 questions (`backend/evaluation/test_questions.json`) couvrant les 6 démarches et des questions hors périmètre.

6.1 Jeu de test

Paramètre	Valeur
Questions totales	10
Questions in-scope	8 (une ou deux par démarche)
Questions out-scope	2 (hors périmètre)
Démarches testées	CNIB, Passeport, Création entreprise, Casier judiciaire, Acte naissance, Cert. nationalité
Script	<code>backend/evaluation/evaluate.py</code>
Résultats	<code>backend/evaluation/results.json</code>

Tableau 7 — Paramètres du jeu d'évaluation.

6.2 Résultats

Métrique	Résultat	Détail
Pertinence du retrieval (in-scope)	100 %	8/8 questions : démarche attendue présente dans les sources récupérées
Non-hallucination (out-scope)	100 %	2/2 questions hors périmètre : formule de repli retournée, aucune invention
Score de pertinence moyen	Métrique cosinus	Scores cosinus : $(1 - \text{dist}) \times 100$ — 0 = aucune similarité, 100 = identique sémantiquement

Tableau 8 — Résultats de l'évaluation du pipeline RAG.

Le taux de pertinence du retrieval de 100 % indique que ChromaDB, avec seulement 32 chunks, retrouve systématiquement la bonne démarche parmi les top-6 résultats. Ce résultat est robuste sur un petit corpus, mais devra être réévalué lors d'une extension à d'autres démarches. Le taux de non-hallucination de 100 % est garanti par le design du prompt et par le modèle Llama 3.1 qui respecte bien la consigne de repli.

6.3 Exemple de réponse évaluée

Champ	Valeur
Question	Quelles pièces sont nécessaires pour faire ma première demande de CNIB ?
Sources récupérées	cnib, passeport, casier_judiciaire (top-6 chunks, métrique cosinus)

Score moyen	Score cosinus significatif — démarche cnib bien classée en top résultats
Retrieval pertinent	OUI — 'cnib' présent dans les sources récupérées
Hallucination	Non applicable (question in-scope)

Tableau 9 — Exemple d'évaluation sur une question CNIB.

7. Limites et perspectives

7.1 Limites actuelles

- **Corpus restreint** : 6 démarches, 6 PDFs, 32 chunks. Toute question hors périmètre est renvoyée vers un message de repli. Une extension requiert sourcing et vérification manuelle de nouveaux contenus officiels.
- **Dépendance aux sources externes** : Si les portails officiels (oni.bf, police.gov.bf, service-public.gov.bf) modifient leurs tarifs ou procédures, les PDFs sources doivent être mis à jour manuellement et l'index régénéré.
- **Modèle d'embeddings généraliste** : paraphrase-multilingual-MiniLM-L12-v2 est un modèle généraliste. Un modèle fine-tuné sur le français administratif burkinabè améliorerait la qualité du retrieval, notamment pour les termes juridiques spécifiques.
- **Dépendance aux plateformes gratuites** : Render redémarre le backend après inactivité (cold start ~30–60 s). La limite RAM de 512 Mo oblige à utiliser l'API HuggingFace distante plutôt qu'un modèle local, ajoutant une latence réseau.
- **Jeu d'évaluation limité** : 10 questions de test suffisent pour valider le principe mais ne constituent pas une évaluation statistiquement robuste. Une couverture plus large avec annotation humaine serait nécessaire en production.

7.2 Perspectives d'amélioration

- **Extension du corpus** : Ajouter permis de conduire, mariage civil, état civil, permis de construire — en réutilisant le même pipeline PDF → index → API sans modification architecturale.
- **Re-ranking** : Ajouter un modèle de re-ranking (ex : cross-encoder) après le retrieval initial pour améliorer la pertinence des chunks sélectionnés avant la génération.
- **Évaluation élargie** : Augmenter le jeu de test à 50+ questions avec annotation humaine de la qualité des réponses, et introduire des métriques comme RAGAS (faithfulness, answer relevance, context precision).
- **Multimodalité** : Permettre à l'utilisateur de soumettre une photo de document pour identification (OCR + classification de la démarche concernée).
- **Déploiement offline** : Packager le système pour fonctionner sans connexion internet, avec un modèle d'embeddings local léger et un LLM quantifié (ex : Ollama + Llama.cpp).

8. Conclusion

GuichetIA constitue une implémentation fonctionnelle et déployée d'un système RAG appliqué aux démarches administratives burkinabè. Le pipeline couvre 6 démarches, 6 PDFs sources et 32 chunks indexés, avec un taux de pertinence du retrieval de 100 % et un taux de non-hallucination de 100 % sur le jeu de test évalué.

L'architecture choisie — ChromaDB (métrique cosinus) + HuggingFace Inference API (embeddings normalisés) + Groq/Llama 3.1 + FastAPI + React — est entièrement gratuite, reproductible par script, et déployée publiquement sur Render et Vercel. Elle illustre comment les outils open source de l'écosystème LangChain permettent de construire rapidement un assistant RAG de qualité sans infrastructure coûteuse.

Les principales limites identifiées (corpus restreint, cold start Render, jeu d'évaluation limité) ont toutes des pistes d'amélioration claires et implémentables dans le cadre d'une version suivante du projet. La structure modulaire du code (ingestion séparée du backend, backend séparé du frontend) facilite ces évolutions.